Customer churn analysis with Delta Lake and Apache Spark
ID2221

March 3, 2026

Jonathan Tadesse

---

1 Introduction

In telecommunications, customer retention is important for profit and long-term growth. It's often more beneficial to retain a customer than acquire a new one. Companies therefore invest resources to analyze customer behavior and prevent churn. This project builds an end-to-end data lakehouse pipeline for analyzing customer churn using the Telco Customer Churn dataset from Kaggle.
The project has two goals: the first is to design a robust, reproducible pipeline to ingest, transform, and organize raw data into an analytics-ready form; the second is to apply a machine learning model to the cleaned data and find insights. The system uses Apache Spark, HDFS, Delta Lake, and JupyterLab, orchestrated with Docker Compose.
This project uses a multi-layered Lakehouse architecture:

Bronze stores raw CSV data, standardized to a schema; **Silver**: we clean data, engineer features, normalize, and reconcile inconsistencies; and **Gold**: we integrate Silver tables into a unified analytical dataset optimized for machine learning.

2 System architecture

Storage (HDFS): A single NameNode and DataNode store both the raw and curated data. The raw CSV files are stored under `/data/raw/telco`, while the Delta tables are organized under `/data/delta/{bronze,silver,gold}` A single NameNode and DataNode store both raw and curated data. We use Delta Lake as the table format, as to give us ACID transactions and schema enforcement.
Compute (Spark): The system has only one spark Master and Worker. All the PySpark notebooks for the different processing layers connect to the cluster through `spark://spark-master:7077`. Spark is configured with Delta extension and HDFS as the default file system.
Development (JupyterLab): A Jupyter environment runs inside the container. It executes all the notebooks associated with each stage in the pipeline process.

3 Data ingestion and bronze layer

Each of the CSV files stored in HDFS are read using Spark's built-in `read` method. For each of the CSV files, we explain what operations we took on them and how we stored them.

Demographics CSV: For the demographics CSV file we investigated the number of `NaN`-values, we observed that there were none across all the columns. We then normalized column names, converted `Yes/No` to `boolean`-values. And finally wrote the DataFrame.

Service CSV: For the Service csv file we did the same procedure as above, we normalized the columns, converted string representations of integers to `int`. We also counted the number of `NaN`-values and observed the following: For `internet type` column we had 1526 missing values, for the column `offer` we had 3877 missing values. We then wrote the table to HDFS.

Population, Location and Status CSV: For the population csv file, we did similar procedures, normalized the columns, converted the column Zip Code to int. We then wrote it to HDFS.

Similar procedures here too, converted `Latitude` and `Longitude` columns to `double`, we checked for `NaN`-values (there were none) normalize the columns and wrote it to HDFS. Then, we normalized the columns of `Status CSV`, counted the number of missing values. We observed 5174 missing values in the columns `churn category` and `churn reason`. We then wrote the table to HDFS.

4 Transformation and Standardization (Silver Layer)

In the silver layer we worked on the tables that we processed in the bronze layer. For the bronze `demographics` table we just removed the `count val` column as it served no purpose. We then wrote it to HDFS and made sure that it was queryable using SQL. For the `location` table we simply dropped the `country` and `state` column as they only had 1 unique entry across both columns. We then wrote it to HDFS and similarly made it queryable using SQL. The `population` DataFrame was already clean in the bronze stage so we didn't have to do anything here, we wrote it to HDFS. For the `service` DataFrame we changed the entries in the `offer` column into numbers same with the internet type, we kept the `null`-values as they are We then wrote it to HDFS. For `status` we also dropped the `count val` column as it had no purpose. We converted the entries `customer status` to numerical values. We then wrote it to HDFS

5 Feature Enrichment and ML Preparation (Gold Layer)

To start with this stage, All tables were loaded from HDFS that were saved after the Silver stage. Next, Some columns were duplicated or some had identical values, and others were not needed for feature enrichment, such as `loaded_at` or `ingested_at`.

Next, the tables were joined. The `Location` table was joined to `Population` by `zip_code`, and the rest were joined based on `customer_id`. At this point feature enrichment was completed and results were saved onto HDFS.

After that, each string type feature was observed and unique values were identified. Then for each string based features, one-hot-encoding was performed by creating new int columns. A key strategic decision was not to reduce number of feature by 1. For example, for each value of gender, 'male' and 'female', two new columns were added instead of just a single column to consider diversity and equity for future data. After that we carefully chose the features and label and ensure no data leakage happens.

6 Machine Learning and Result:

Next, the whole dataset was split into train (70%) and test (30%) data. Spark ML was used for machine learning. **VectorAssembler** were used to create a new feature column combined from all the numerical feature. Also **StandardScaler** api was to normalise the features. Finally, **LogisticRegression** api was used to run Logistics Regression on to the train dataset and then test data set was used to evaluate the model. **BinaryClassificationEvaluator** and **MulticlassClassificationEvaluator** apis were to evaluate accuracy, F1, AUC under ROC, Precision and Recall.

The analysis of *coefficients* revealed that the feature most contributing to customer churn status determination is *number_of_referrals* and *tenure_in_month*. The model achieved accuracy of 84.5% and F1-score of 84.26%.

More details metrics can be found in the table 2 . The confusion matrix can be found in table 1.

Table 1: Confusion Matrix

	Pred. Neg.	**Pred. Pos.**
Actual Neg.	1345	132
Actual Pos.	179	351

Table 2: Evaluation Metrics

Metric	**Value**
Accuracy	0.845042
Precision	0.726708
Recall	0.662264
F1-score	0.842661
AUC-ROC	0.901225

7 Appendix

7.1 Project Setup:

The project contains `docker-compose.yml` file that setups the docker container. The docker installation is a prerequisite to run the project. After that run the following command to setup Docker.

```
docker compose pull
docker compose up -d
```

Once the docker containers are running, a series of commands needs to be run in order to setup user permission, creating folders, and copying the required files onto HDFS. Those commands can be found in `Readme.md` file.

The whole project is divided into three services: Bronze, Silver and Gold. Each service has separate notebook and a python file. The python file are the same code from the notebook extracted so we can run the code with `spark-submit` command that utilizes spark service employ multiple worker if setup like that. But for running the code or experimentation Jupyter Notebook can be run as well.

The command that runs the pipeline are given below:

```
docker exec -it spark-master /opt/spark/bin/spark-submit --master
  ↪ spark://spark-master:7077 --deploy-mode client --packages
  ↪ io.delta:delta-spark_2.12:3.3.1 --conf spark.jars.ivy=/tmp/.ivy2 --conf
  ↪ spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension --conf
  ↪ spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog
  ↪ --conf spark.hadoop.fs.defaultFS=hdfs://hdfs-namenode:9000
  ↪ /opt/project/data/jobs/{bronze/bronze_services.py,
  ↪ silver/silver_services.py,gold/gold_service.py
```

Separately, Jupyter notebook can be used to run the codes as well. The URL for Jupyter notebook is `localhost:8888`. When opened there should be a `work` folder inside which each stages of Jupyter Notebook can be found.